

#### 4.4 Design of neural based computational lab

Here, we first give a brief and general introduction on neural networks, and then explain what role they play in our modeling.

Neural networks are conceptually quite simple computing systems, inspired by structure of biological neurons in human brain. Neural networks use computational methods to “learn” information from data without relying on a predetermined equation as a model. Deep learning is especially suited for image recognition, which is important for solving problems such as facial recognition, motion detection. “Such systems progressively learn through examples without task specific programming. Various architectures of ANNs are being used of which Multilayer Perceptron (MLP) is the most popular one. MLP consists of an input layer, a hidden layer and an output layer of neurons. While the neurons in the input layer are typically linear, the neurons in the hidden layer have nonlinear activation functions. The neurons in the output layer may be linear or nonlinear. The input layer consists of input neurons with associated weight. The hidden layer receives the signals from the input layer. There’s also a threshold parameter known as bias which is associated with each neuron in the hidden and output layers” [83]. “An elementary neuron with  $R$  inputs is shown below. Each input is weighted with an appropriate  $w$ . The sum of the weighted inputs and the bias forms the input to the transfer function  $f$ . (Fig. 48)” [84]

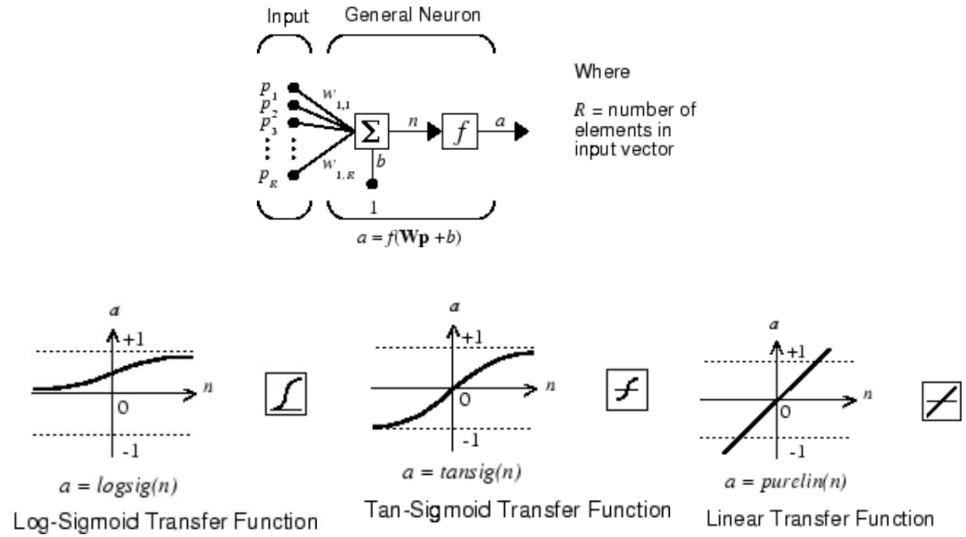
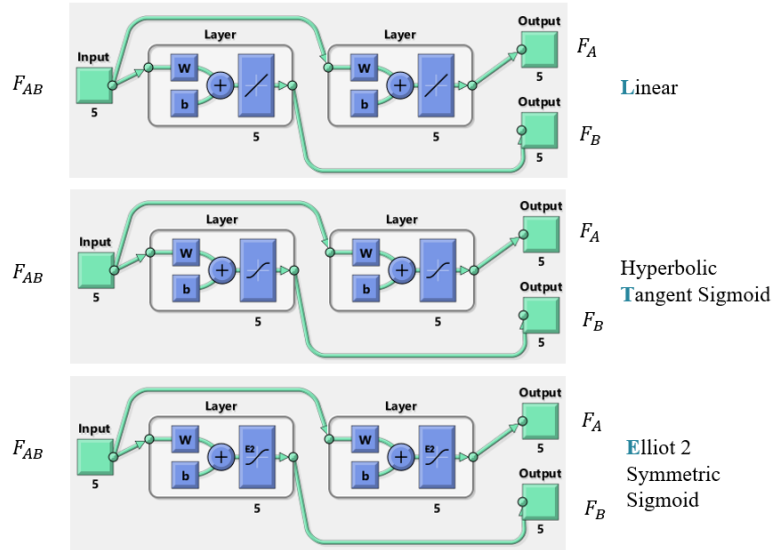


Figure 48. The structure of a general neuron as foundational unit of neural networks [84]

Development of three stable neural network unit featuring vanished gradients to serve as a computational laboratory:

Three neural network unit with three different transfer functions (i.e. linear, hyperbolic tangent sigmoid, and elliptic symmetric sigmoid) were trained by the sets of Fibonacci sequences with the configurations illustrated in Figure 49. (The ABT is chosen to exemplify the NN details).



- All NN's are composed of an input layer that receives  $F_{AB}$ , 2 hidden layers each with 5 neuron designated to predict  $F_A$  and  $F_B$  separately.
- **ABL**, **ABT**, **ABE** are 3 NN's trained based on **L**inear, **T**angential and **E**lliptical transfer function, respectively.

- There are **60** elements in ABT.
- **50** elements in terms of weight ( $w$ ) (i.e. one matrix of  $[ ]_{5 \times 5}$  for each layer).
- And, **10** elements in terms of bias ( $b$ ) (i.e. one vector of  $[ ]_{5 \times 1}$  for each layer).
- For example, the following numerical configuration, is resulted upon training the "ABT" neural network using Levenberg-Marquardt backpropagation algorithm with aim of minimizing sum of squared errors in predicting  $F_A$  and  $F_B$ .

|       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|
| -1.42 | 2.51  | 1.77  | 1.28  | 1.35  | -0.81 |
| -0.61 | 0.48  | -3.08 | 0.94  | -1.98 | 3.09  |
| -2.01 | -2.19 | -2.27 | -0.24 | 0.93  | 2.89  |
| -2.20 | -1.59 | -1.98 | 0.28  | -1.89 | 2.72  |
| 3.13  | -1.04 | 1.06  | -1.44 | -0.93 | 1.55  |

|       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|
| 1.43  | -0.15 | 0.12  | 3.54  | 0.57  | -4.69 |
| 2.42  | -0.55 | 2.75  | 0.28  | 1.06  | -3.94 |
| -2.98 | -0.38 | 2.10  | -1.06 | 0.62  | 0.85  |
| 2.30  | -1.04 | 0.77  | 2.35  | -1.56 | -0.44 |
| 2.04  | 0.06  | -0.90 | 2.79  | -1.47 | 0.67  |

The first layer in the figure connected to output  $F_B$

The second layer in the figure connected to output  $F_A$

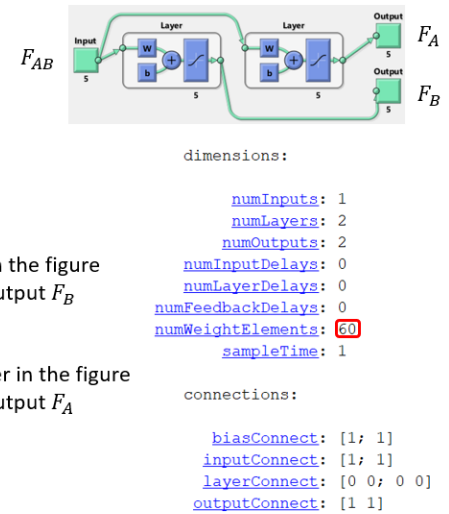


Figure 49. The detailed configuration of ABT (i.e. the neural network set up using hyperbolic tangent as transfer function)

Following the training, the neural network units shows properties of a bounded function with respect to the inputs (i.e.  $F_{AB}$ ), meaning for all the input they return identical vectors of :

$$F_A = [1, 1, 1, 1, 1]; F_B = [1, 1, 1, 1, 1];$$

This seemingly short circuited configuration provides critical information on the hollistic state of data distributions. Specifically, the weights and biases calculated upon the training indirectly capture the distribution of input sets with respect to their corresponding output set. This can be viewed as an unconventional method of averaging distributions.

We will show how this particular quality can form a structure preserving map, and yield a commutative diagram across the three different neural network units.

Following our strategy to cover broad ranges of possibilities with respect to variation potentials (vertical, horizontal, and cross variations) The following scheme was arranged:

- Three different fractions of  $F_A$  (i.e. 0.4, 0.7 and 1) are considered.
- The input to ABL is in form of  $[0.4 * F_A, 0.7 * F_A, F_A]$  while the input to ABT and ABE are in form of  $[\frac{1}{0.4 * F_A}, \frac{1}{0.7 * F_A}, \frac{1}{F_A}]$ .

Defining the quantitative measures reflecting relative deviations from:

$$F_A = [1, 1, 1, 1, 1]; F_B = [1, 1, 1, 1, 1];$$

is done with a combinatory approach using “trace” of matrices as scores. For each run of  $[0.4, 0.7, 1]$  vs  $[L, T, E]$  (i.e. 9 pairs) two groups of calculation is done:

1. Changes in  $F_B$  with respect to  $F_A$ , and the  $F_{AB}$
2. Changes in  $F_A$  with respect to  $F_B$ , and  $F_{AB}$ .

It is important to notice that “changes in  $F_A$  and  $F_B$ ” exactly mean their deviations from identity vectors  $F_A = [1, 1, 1, 1, 1]; F_B = [1, 1, 1, 1, 1]$ .

Another important issue is that the neural network units are trained with the input set of  $F_{AB}$  where  $F_{AB} = F_A + F_B$  however when we feed the

input to the neural network to monitor its changes, our input set is actually  $F_{AB} = F_A$  because we do not have information about  $F_B$ . That said, the NN's behavior is unaffected and they will “act” as they are receiving some sort of  $F_{AB}$  that is constructed as  $F_{AB} = F_A + F_B$ . This subtle issue enables us to observe the effects of a specific  $F_A$  (i.e. an inquiry) on  $F_B$  and quantify its relation with respect to all or nearly all elements or aspects of the process.

As for the quantification method and scoring system the following matrix operation is established. All matrices have the dimension of  $[ ]_{5 \times 5}$ .

- $[B * A^T], [dB * A^T], [dB * A_{in}^T]$ , (reflecting the changes of B to A) (67)
- $[A * B^T], [dA * B^T], [A_{in} * dA^T]$ , (reflecting the changes of A to B) (68)

When the trace of the three matrices are summed up in each group, they yield a Figure 50 scores such as below.

|        |     | L    | T     | E    |
|--------|-----|------|-------|------|
| B to A | 0.4 | 1.05 | 3.94  | 3.30 |
|        | 0.7 | 1.67 | 3.64  | 3.00 |
|        | 1   | 2.14 | 3.52  | 2.90 |
| A to B | 0.4 | 1.12 | -1.28 | 3.43 |
|        | 0.7 | 1.71 | -1.12 | 2.94 |
|        | 1   | 2.10 | -1.04 | 2.70 |

Figure 50. Scores calculated from traces of matrices defined by Equations 67&68

Conceptually this figure summarizes the changes along 4 axioms: path, scale, AtB and BtA. Each score in table 6 is the sum of traces for three matrices ( $\sum_3 [ ]_{5 \times 5} \text{ trace}$ ) in aforementioned groups, it can be seen that the information in table 6 is obtained from traces of 54 matrices  $[ ]_{5 \times 5}$  (i.e. 3 scale\* 3 path\* 6 measure = 54 matrices that overall consist of 1350 numerical value, each probably capturing an infinitesimal relation